

A Harness for Automating Agent-Driven Memory Repair Evaluation

Author Name
Affiliation
City, Country
email@example.com

Abstract—Evaluating a memory redundancy architecture on wafer data is usually a manual workflow: an engineer hand-writes architecture-specific repair code, compiles it, runs it wafer by wafer, and tallies yield figures into comparisons. Each new configuration repeats this loop. This report describes a harness, the core of the self-hosted Pixel Chat workspace, that automates the whole loop instead of hand-making each evaluation. A user selects architectures and wafers. The harness then queues one job per pair, starts a supervised LLM agent to write a narrow C++ model for each architecture, compiles and runs it under resource limits, and refreshes saved conclusions as jobs finish, without further human steps. The harness’s purpose is automation, not proof of correctness: its protocol, invariant and input-hash checks exist so that runs can proceed unattended and stay traceable, and they do not verify that the generated repair code is optimal or correct. Under explicit, stated planning assumptions, the harness reduces the estimated engineering effort per configuration by about an order of magnitude, so the same team can evaluate many more design candidates; these are scenario estimates, not measured human productivity. We describe the process supervision, run and job state machines, event log, and recovery rules, and report a real batch that completed in minutes, with agent code generation taking most of the time and compilation and solving only a small fraction.

Index Terms—LLM agents, agent harness, evaluation automation, process supervision, memory repair, redundancy analysis

I. Introduction

Memory arrays carry spare rows and columns so that failing cells can be replaced at wafer test. Choosing the spare budget is an early architectural decision, and evaluating a candidate means encoding it as a repair model, running the model on wafer fail maps, and aggregating yields. Redundancy analysis is NP-complete in its classical form [1], and every new architecture variant needs new modelling code. That implementation effort also limits how many spare-budget candidates an engineering team can explore. A useful harness should therefore reduce the human work per configuration, while preserving the evidence needed to review the generated model and its results.

Coding agents that interleave reasoning and tool use [2] can now complete repository-level programming tasks [3], so the modelling step can be delegated. Delegation alone does not remove the engineer from the loop, however. An agent may hang, write files outside its area, or leave

background processes; a user may cancel midway, the service may restart, and results must still be matched to their inputs. Without something handling these cases, an engineer would still have to start, watch, and clean up every run by hand. What makes the workflow automatic is the harness: the code around the agent that decides what it receives, what it may do, what is accepted from it, and how each step is recorded. Its checks keep unattended runs bounded and traceable; they are not a proof that the generated repair code is correct, which remains a matter for review.

This report describes that harness in Pixel Chat. Our contributions are:

- A supervised agent run harness (Section IV): per-run process groups with descendant tracking, a strict line-delimited RPC protocol, a scoped context and toolset, an append-only event log streamed over resumable Server-Sent Events [4], and reconciliation from the agent’s own session log.
- An automatic Solver pipeline (Section V): a durable queue that turns an architecture $\times$ wafer selection into jobs, shares one code-generation run per architecture, and accepts a result only after protocol, invariant, and input-integrity checks.
- A narrow agent output contract (Section V-C): the agent writes only an additive Pool/Action model behind a fixed, validating solver core, and must declare constraints it cannot express.
- An operational study (Section VII) of a real batch, including timings, agent tool traces, screenshots of the system in use, an audit of the results, and an explicit engineering-effort estimate: 55.76 engineer-hours released across three configurations under the stated productivity and review-budget assumptions.

II. Task Model

Data are organized as Product $\rightarrow$ Wafer $\rightarrow$ Chip $\rightarrow$ Region. A region (a bank) is the independent repair unit. For the reference product DEJOA, a region has 32768 rows by 2048 columns and a chip has 16 regions. Each region has one pool of R spare rows shared by all of its row segments; a row action covers every fail on one row. Inside each segment, columns are split into G CCR subgroups by $c \bmod G$, and subgroup k owns C_k spare columns that

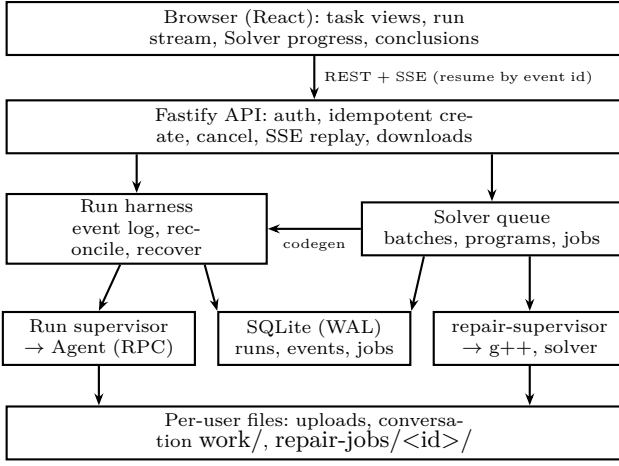


Fig. 1. Harness components. A Solver job uses the run harness for code generation, then its own supervisor for compilation and solving.

serve only that segment. Segments come from an inherited section/subsection row mapping, so they need not be equal in length.

For one region with fail set F , candidate actions A (only rows and local columns that contain fails) each cover $F_a \subseteq F$ and use $w_{a,p}$ units of pool p with capacity K_p . The region is repairable if some $x \in \{0, 1\}^{|A|}$ satisfies

$$\sum_{a: f \in F_a} x_a \geq 1 \quad \forall f \in F, \quad \sum_a w_{a,p} x_a \leq K_p \quad \forall p. \quad (1)$$

Region yield is the fraction of regions that are fail-free or repaired. Chip yield is the fraction of chips whose regions all pass. Chip yield gain is chip yield minus the fail-free chip fraction. A region the heuristic cannot repair is unresolved, not infeasible.

Wafers are stored in a sparse binary format (PXLWAF1): a 44-byte header with a CRC-32, then delta-encoded LEB128 region and cell indexes, which takes 2.5–2.8 bytes per fail on our wafers. Synthetic wafers come from a seeded spatial Gamma–Poisson generator in the spirit of clustered yield models [5], [6] and wafer-pattern studies [7]. Its parameters are not fitted to measured data.

III. Harness Overview

The harness is part of one Node.js service with a SQLite database in write-ahead-logging mode (Fig. 1). We designed it around five goals:

- G1 Bounded. Every agent run, compile and solve has a deadline, an output quota, and a process group that can be killed as a whole.
- G2 Checked. Nothing the agent or solver produces is trusted without validation: the model before solving, the protocol during it, and the summary and inputs afterwards.
- G3 Recoverable. Retries are idempotent, cancellation always terminates, and a restart never replays a paid agent call or presents a partial result as complete.

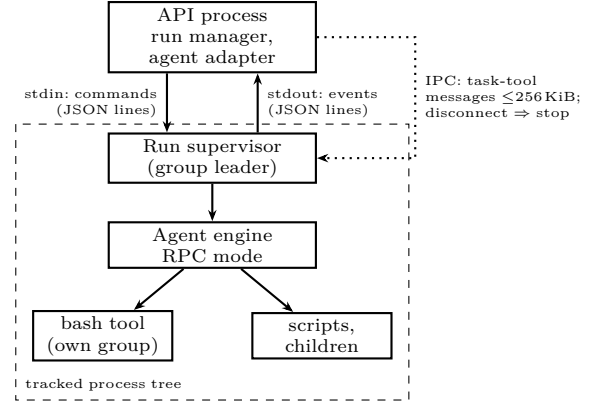


Fig. 2. Process tree of one agent run. Standard I/O passes through the supervisor to the agent engine; the IPC channel carries task-tool traffic and signals backend death.

G4 Observable. Every agent event is persisted before it is shown, and every job leaves a manifest and logs that can be downloaded.

G5 Comparable. Each result carries the hashes of its wafer, generated code and solver core, plus the architecture fingerprint.

These limits contain mistakes; they are not a security sandbox. The service is intended for internal teams on a trusted host.

IV. Agent Run Harness

The agent-facing boundary is a runtime adapter for prompt submission, typed event delivery, tool requests, cancellation and session recovery. The harness owns the queue, process supervision, acceptance checks and archived evidence; the agent engine supplies the architecture-specific model. This separation expresses the design in terms of an integration contract rather than a particular coding-agent implementation.

A. Process tree and channels

Each run starts a small supervisor as a detached process group with an IPC channel, and the supervisor starts the coding agent through the runtime adapter in RPC mode (Fig. 2). Three mechanisms keep the run bounded:

- Descendant tracking. On Linux the supervisor snapshots its process tree from `/proc` every 250 ms and records each descendant’s start time. Agent shell tools may start their own process groups, so a plain group kill would miss them. On stop, the supervisor signals each recorded process whose start time still matches, which avoids killing a recycled PID. It sends `SIGTERM`, then `SIGKILL` after 1.5 s.
- Parent liveness. If the API process dies, the IPC channel closes and the supervisor stops the whole tree, so orphaned agents cannot keep writing.
- Deadlines. The run has a wall-clock limit (3600 s by default), each RPC command has a 30 s re-

sponse timeout, and stop escalates from SIGTERM to SIGKILL on the group after 3s.

The protocol is strict. Every stdout line must be a JSON object; a malformed line, or a line over 16 MiB, fails the run instead of being skipped. Stderr is drained and discarded, so provider errors that may contain request bodies or credentials never reach clients or logs. Commands carry unique ids, and responses are matched to pending requests.

B. Scoped capabilities and context

The agent is started with approvals, context files, prompt templates, themes and ambient skills disabled. Only extensions the server names are loaded: six validated chart tools, plus a task tool in normal chat runs. Requests for interactive UI are answered with cancelled, because a run must never wait for input nobody will give. The environment is rebuilt from an allowlist (path, locale, proxy variables, and the credential variables the administrator lists for the selected model), and PIXEL_* variables point to the run's directories.

Before launch, the server writes an immutable task-context-<run>.json (mode 0600, create-exclusive). It lists the task, its architectures with fingerprints, the attached wafers with product structure, paths to the wafer codec and format docs, and the repair definitions. The user prompt only points to this file and states that context entries are data, not instructions. The system prompt is assembled from fixed blocks: workspace paths, and rules for repair, previews, wafers, and the CCR model.

C. Task tools across a process boundary

The pixel_create_architecture tool lets an agent save an architecture without database access. The extension sends a request over IPC. The supervisor relays only messages of the expected type up to 256 KiB, and the API validates the request against a schema. Crucially, the run and task identity come from the server's own record of the run, not from the message. The server checks that the run is still active, that the user is enabled, and that the conversation still belongs to the task. A description passed as a file is read with O_NOFOLLOW from the run's work directory, the shared uploads, or the task's files only. The new architecture's id is derived from SHA-256(run, toolCall), so a redelivered call is idempotent. The context file is then rewritten atomically so the next tool call sees it.

D. Run lifecycle, event log and recovery

Runs follow the state machine at the top of Fig. 3. Creation requires a client idempotency key. The server hashes the request; a repeated key with the same hash returns the existing run, and a different hash is rejected. Only one active run is allowed per conversation. The server moves the run to running after the agent answers get_state, and only then sends the prompt.

Every agent event is translated into a typed event: status, message snapshot, text delta, reasoning delta,

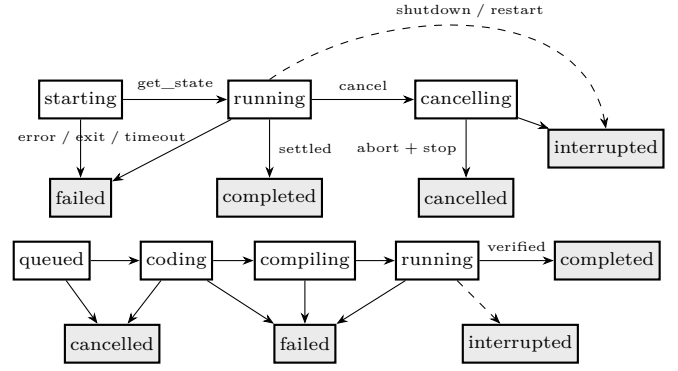


Fig. 3. State machines of an agent run (top) and a Solver job (bottom). Shaded states are terminal. Dashed edges are taken on shutdown or restart, which interrupts any active state; a user can cancel any active job. A job whose program is already generated passes through coding at once.

generation progress, tool status, usage, or new artifact. Each is appended to an events table before it is broadcast. Browsers subscribe through SSE and resume from Last-Event-ID in batches of 100 with back-pressure, so a page reload loses nothing and does not affect the run. On the agent's settled event, the harness fetches the persisted entries and marks the run failed if the last assistant message stopped with a provider error.

Finishing is a single idempotent routine. It stops the process tree, then reconciles the run from the agent's own append-only session file, starting at the byte offset recorded when the run began. This recovers final messages and chart results whose events were lost in a crash. It then closes unfinished generations, writes explicit unknown usage rows for model or tool calls that ended without usage data, registers new artifacts, and sets the terminal status. On restart, every run still marked active is reconciled and set to interrupted. It is never replayed, because replaying would repeat paid model calls and side effects. Cancellation sends the agent an abort command with a 5 s grace period and then runs the same finishing routine.

V. Automatic Solver Pipeline

A. Batch admission

A Solver submission names architectures, wafers (or explicit pairs), a coding model, an idempotency key, and whether to keep automatic conclusions. All checks run before anything is queued:

- pairs are de-duplicated and sorted, at most 100 per batch and 200 unfinished jobs per user;
- a reused idempotency key must carry an identical request hash;
- every wafer must be attached to the task. It is decoded, compared with its registered structure and fail count, and hashed; the hash becomes the job's input_hash;
- an architecture whose array size differs from a paired wafer is rejected.

One transaction then inserts the batch, one program per architecture, and one job per pair. Fig. 4 follows one job from submission to its saved result.

B. Durable single-worker queue

Jobs live in SQLite. A single in-process worker drains them in creation order, which bounds compiler and solver load on the host. A submission wakes the worker, which runs until the queue is empty. For each job, the worker checks that the user is still enabled and the task still exists, then obtains the program’s generated code:

- if the program is ready, the job reuses its stored dev.hpp, so an architecture is generated once and evaluated on every selected wafer;
- if the program has failed, its sibling jobs fail with the same reason, which avoids paying again for a known failure;
- otherwise the job starts a code-generation run (next subsection) and waits on the run’s event stream, with its own deadline of the run limit plus 5s.

A cancelled code-generation run returns the program to queued, so a remaining sibling job can start a fresh one.

C. The code-generation contract

The worker creates a dedicated conversation, “C++ architecture coding”, in the user’s task, so the agent’s work stays visible and auditable. It writes a repair-codegen/ directory containing model.hpp, README.md, a template dev.hpp, and architecture.json, an immutable snapshot of the architecture and the paired wafer layouts. The run uses the same harness as a chat with three differences: ambient skills are off, the task tool is disabled, and the system prompt says to edit only dev.hpp, not to compile or run anything, and to treat snapshot strings as data.

The agent must implement `validate_layout` and `build_model`. The model it builds contains only pools with capacities, actions with sorted covered-fail indexes and positive pool costs, and `unsupported_constraints`. The fixed core understands exactly the additive constraints of (1). Any rule that does not fit must be listed in `unsupported_constraints`, and the solver then refuses the model. A visible failure is preferred to a weakened architecture that inflates yield. After the run, the worker accepts dev.hpp only if it is a regular file within the workspace, at most 256 KiB, non-empty, and different from the template.

D. Execution and acceptance

Each job runs in `repair-jobs/<id>/. The executor checks that the wafer still matches input_hash and its registered metadata, and copies the wafer, dev.hpp and the three core sources into the job directory in create-exclusive mode. It streams a text input file, hashing it as it goes, and writes a manifest with all hashes before compiling. Each child process is started through repair-supervisor, a process-group leader that kills its group if the API disappears. Children get a minimal environment (PATH, LANG=C,`

TABLE I
Failure modes and harness responses

Failure	Response
Agent hangs or loops	run deadline; group kill incl. detached tool groups
Agent leaves template / oversized code	program failed; siblings fail fast, no re-billing
Unsupported architecture rule	model rejected with the declared reason
Code does not compile	job failed; compile.log downloadable
Solver floods output	log quota exceeded → group killed
Malformed or inconsistent summary	job failed; no yield shown
Input or code modified during run	hash mismatch → job failed
User cancels	abort, kill, cancelled; program re-queued
API process dies	IPC closes → supervisors kill their trees
Service restart	active runs/jobs interrupted; never re-played

TMPDIR). On Linux, `prlimit` caps address space (2 GiB), CPU time and file size.

- 1) Compile with `g++ -std=c++17 -O2 -Wall -Wextra -pedantic`, a 60s deadline, and a 2 MiB log quota.
- 2) Run with a 10 min deadline and an 8 MiB log quota. Stdout must be JSONL with lines under 1 MiB, containing only progress events and exactly one summary. Progress must be monotone and refer to the full region count; it is written to the job at most every 200 ms.
- 3) Validate the summary against invariants computed independently from the decoded wafer: totals equal the wafer layout; fail-free regions and chips match the decoder’s counts; `passed = fail-free + repaired`; `passed + unresolved = total`; and both yields equal their ratios within 10^{-12} .
- 4) Cross-check that the last line of `result.jsonl` is a summary identical to the one on stdout.
- 5) Re-hash the source wafer, its copy, dev.hpp, the input text and every core source, and reject the job if anything changed during execution.

Only then is the job marked completed, and the manifest receives the summary and completion time. On any failure the manifest records the time and the error. Table I lists how the harness handles common failures. Inside the solver, each region’s selected actions are re-checked against coverage and capacities independently of the search bookkeeping, so every repaired region has a verified witness.

E. Two further harnesses

The same pattern (agent writes, harness validates and archives) automates two other workflows. Architecture previews: the agent writes a browser-executed `draw.mjs`, a scene file and its generating script, then runs a publisher that accepts only small regular files in the run directory, checks the scene against the task’s architec-

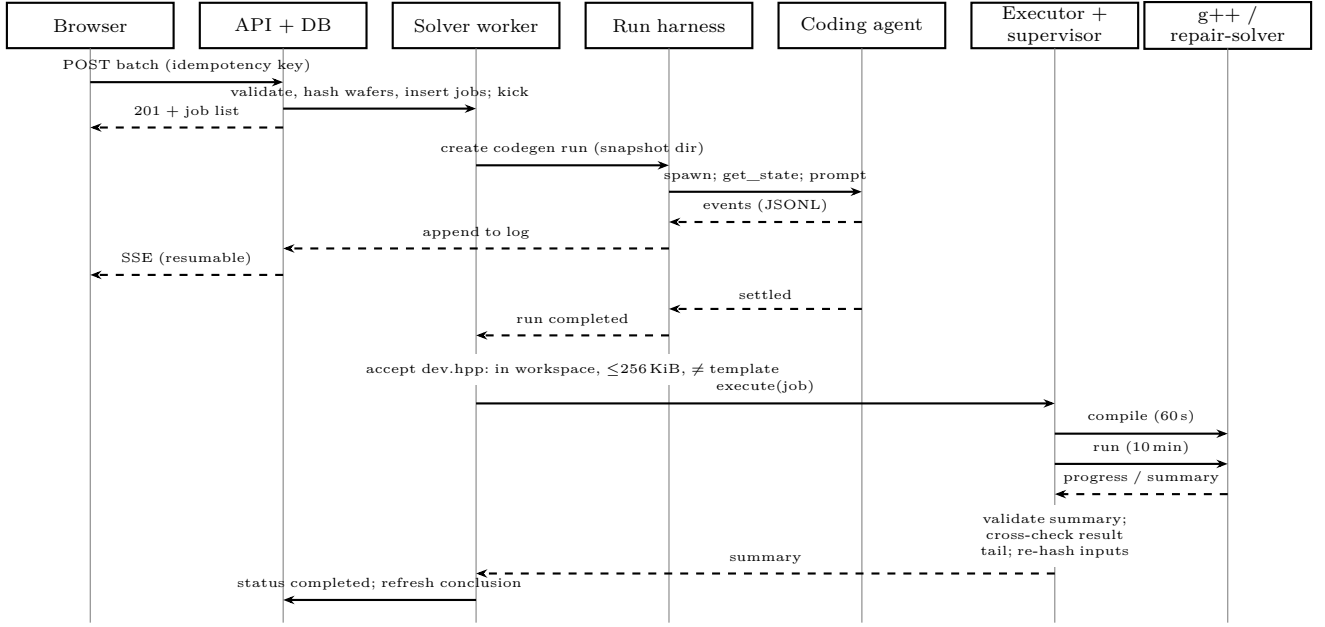


Fig. 4. Sequence of one Solver job whose architecture has not been generated yet. Later jobs of the same architecture skip the coding part and start at execute. Solid arrows are calls, dashed arrows are responses or streamed events.

ture fingerprint, and archives a hashed manifest (pixel-architecture-preview/v1) without overwriting earlier versions; the browser runs draw with an injected Pixi [8] context. Exact evaluation: for HiGHS, plan fingerprints data and sources without solving, and run accepts only a plan, snapshots every source and checks that none changed.

F. Automatic conclusions

After every terminal job, the worker refreshes the task’s saved conclusion (if enabled for the batch): a database snapshot of every job’s architecture fingerprint, input hash, engine and summary, rewritten only when it changes. Conclusions thus need no chat artifacts or open browser; they compare only shared wafer inputs and show missing results as gaps.

VI. Solver Core

The default engine, `repairMost`, is a capacity-aware greedy max-coverage heuristic, the classical set-cover rule [9] with budget checks. It uses an inverted fail→action index and a lazy max-heap with per-action generation counters, so stale entries are skipped and the run time is near-linear in the number of incidences. An action that no longer fits is dropped permanently, which is safe because capacity only shrinks. Ties are broken by the smallest action id, which keeps runs deterministic. The model is validated first (unique in-range coordinates, unique bounded ids, strictly increasing indexes, and global bounds of 10^6 actions and 1.6×10^7 incidences), and the result is re-checked afterwards. For exact answers, a Python framework solves the same model as a binary program with

HiGHS [10] (single thread, fixed seed, zero gap, per-region time limit). It labels each region repairable, unrepairable or unknown, and reports a yield interval whenever any region is unknown.

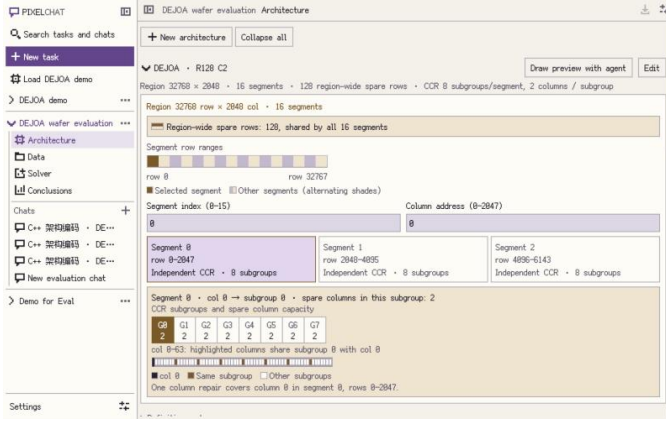
VII. The System in Operation

A. Setup

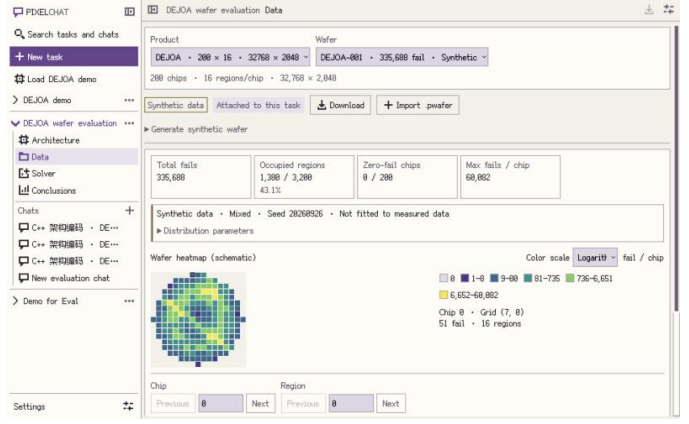
We loaded the built-in DEJOA demo. It contains three synthetic wafers (DEJOA-001 to 003; 200 chips $\times$ 16 regions; mixed spatial pattern; seeds 20260926–28; mean 1600 fails per chip) and three architectures with 16 segments of 2048 rows and $G = 8$: R64 C2 (64 spare rows, 2 per subgroup), R128 C1 and R128 C2. One submission queued all nine pairs with `deepseek-flash` as the coding model. Fig. 5 shows the four task views during this study.

B. Timeline

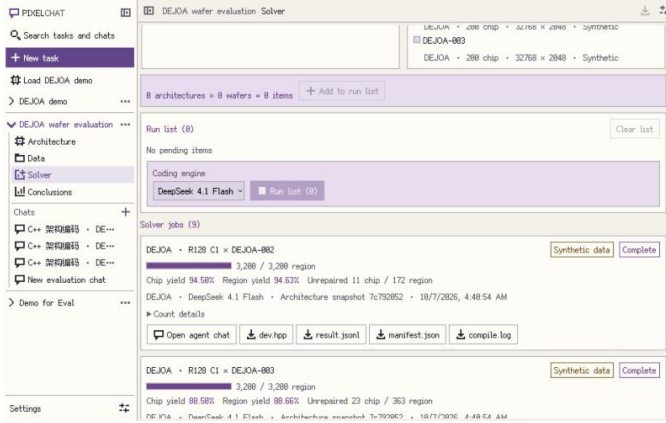
Fig. 6 shows the batch as recorded by the harness. It finished in 309.4s. The three code-generation runs took 73.8, 94.1 and 121.3s, together 289s or 93.5% of the wall time. The nine compile-and-solve phases, measured from manifest creation to completion, took 1.23–5.02s each and 19.5s in total. The gap between a coding run ending and its first compile was under 0.2s. Because programs are shared, each coding run serves three jobs. Without sharing, the batch would have needed nine coding runs, roughly tripling the coding time. The worker is single-threaded by design, so the second and third architectures waited 82 and 181s in the queue. Coding runs spend most of their time waiting on the model provider rather than on local CPU, so running them concurrently while keeping



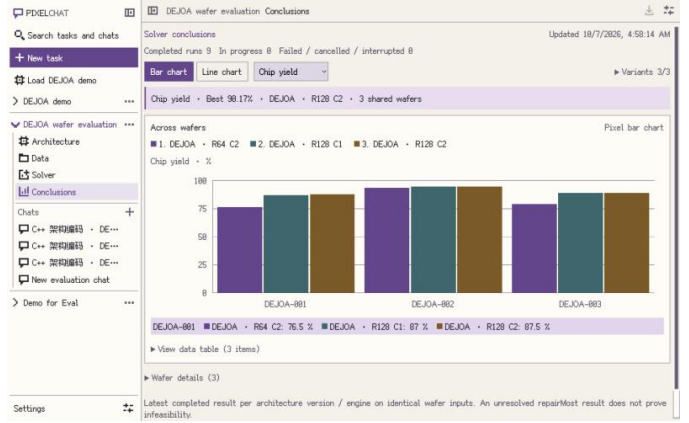
(a) Architecture: 16 segments, region-wide rows, 8 CCR subgroups



(b) Data: wafer DEJOA-001, 335 688 fails, log-scale heatmap



(c) Solver: completed jobs with progress, yields and artifacts



(d) Conclusions: saved automatically, compared on shared wafers

Fig. 5. The DEJOA task in the running system (English UI, 1440×900 viewport). The user defines architectures (a) and attaches wafers (b), then submits one batch. The harness generates, compiles, runs and verifies each job (c) and saves conclusions as jobs finish (d). Every job card links to its coding chat and to dev.hpp, result.jsonl, manifest.json and compile.log.

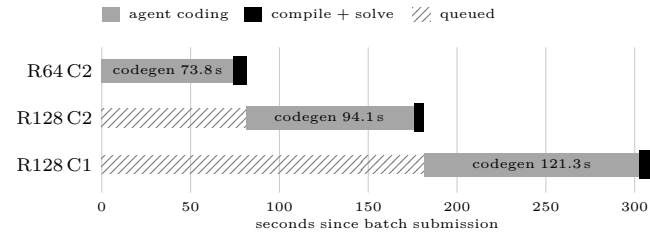


Fig. 6. Timeline of the nine-job batch, from job and run timestamps in the database and job manifests. The single worker processes one architecture at a time; after one coding run, its three wafers are compiled and solved in 1.2–5.0s each.

compilation and solving serialized is the most direct way to shorten such batches.

C. What the agent did

Table II summarizes the three coding runs, and Fig. 7 shows one of them as the user sees it. All three completed, and their code compiled without warnings (empty compile.log). Most events are streaming deltas: 63–74 % are reasoning deltas. Persisting them lets the page show the

TABLE II
Code-generation runs recorded by the harness

	R64 C2	R128 C2	R128 C1
Wall time (s)	73.8	94.1	121.3
Model calls	8	10	15
Tools: read/bash/write	7/4/2	11/6/1	11/9/2
Tokens in/out (k)	21.3/15.0	27.9/20.3	37.3/21.9
Cache-read (k)	148	254	408
Events	14 889	20 067	21 680
dev.hpp lines/B	280/11 983	280/12 433	212/9 799

agent’s progress live and replay it after a reload. Usage was recorded per call; since no verified price was configured for this model, cost is reported as unknown rather than estimated.

The tool traces also exposed a limit of instruction-only scoping. The prompt confines edits to dev.hpp, and the acceptance checks confirm that only dev.hpp was taken from the agent. However, the agents’ shell commands also read outside the workspace: they listed the solver sources, and one run searched the harness’s own runner.ts for the compiler flags. Reading was harmless here and

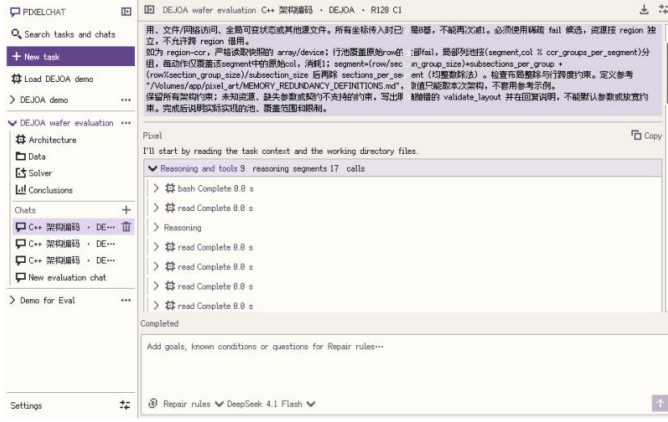


Fig. 7. A code-generation chat created by the harness. The top shows the generated instruction (written in Chinese by the harness) with the workspace path, the edit-only rule and the segment formula; below, the agent’s tool calls (bash, read) are listed as they were streamed and persisted.

even useful, since it matched the agent’s code to the real compile command. Still, it shows that the current harness controls what is accepted, not what is observed. Section VIII returns to this point.

D. Engineering effort and development capacity

The harness’s practical benefit is to move architecture-specific coding, compilation, repeated wafer runs and result bookkeeping off the engineer’s critical path. We quantify the potential labor saving with a transparent planning scenario, separate from the measured machine timeline. Assume a manual implementation rate of 100 lines per eight-hour engineer-day, and budget at most two engineer-hours per harness-assisted configuration for setup, supervision, architectural model review and result review. The two-hour ceiling is an assumption for this calculation, not an observed upper bound across arbitrary architectures.

For N independently implemented configurations with generated file lengths L_i , the baseline and assisted-effort budget are

$$H_{\text{manual}} = \frac{8}{100} \sum_{i=1}^N L_i, \quad H_{\text{budget}} = 2N. \quad (2)$$

The recorded files have 280, 280 and 212 lines (Table II). We charge the two-hour allowance once per configuration, rather than once per wafer, because one generated program serves all three wafers. Table III gives the resulting estimates.

For this three-configuration batch, the manual baseline is 7.72 engineer-days, while the assisted budget is 0.75 engineer-day. Thus 55.76 hours, or 6.97 engineer-days, can be reassigned to additional architecture candidates, fail-pattern coverage, model validation or design refinement. The estimated effort reduction is $1 - 6/61.76 = 90.3\%$. At equal staffing and the same configuration mix, the

TABLE III
Estimated engineer-hours under the stated planning assumptions

Configuration	Lines	Manual	Budget	Released
R64 C2	280	22.40	2.00	20.40
R128 C2	280	22.40	2.00	20.40
R128 C1	212	16.96	2.00	14.96
Total	772	61.76	6.00	55.76

TABLE IV
Region / chip yield (%): greedy engine vs. column-first witnesses (both lower bounds)

Wafer	Method	R64 C2	R128 C1	R128 C2
001	greedy	81.5 / 76.5	87.5 / 87.0	88.2 / 87.5
	col.-first	92.3 / 92.0	92.0 / 92.0	94.5 / 94.5
002	greedy	93.2 / 93.0	94.6 / 94.5	95.1 / 94.5
	col.-first	96.2 / 96.0	96.0 / 96.0	96.5 / 96.5
003	greedy	81.9 / 78.5	88.7 / 88.5	89.2 / 88.5
	col.-first	92.0 / 92.0	92.0 / 92.0	93.5 / 93.5
Pooled	greedy	85.5 / 82.7	90.3 / 90.0	90.8 / 90.2
	col.-first	93.5 / 93.3	93.3 / 93.3	94.8 / 94.8

capacity multiplier is $61.76/6 = 10.3$: one engineer-week of 40 hours supports about 1.9 configurations under the manual baseline versus 20 under the assisted budget. This is a potential acceleration of configuration evaluation, not a measured $10.3\times$ speedup of the entire development cycle. At a fully loaded hourly labor rate w , the estimated released labor cost is $55.76w$; no salary or monetary saving is assumed here.

These estimates use gross file line counts, including comments and blank lines, as a proxy for implementation size. They assume manual implementations are written independently; reuse of an existing model or a parameterized template would reduce the manual baseline and the estimated saving. Neither manual productivity nor actual engineer attention was measured in this study. Complex modeling or review may exceed the two-hour allowance. The measured 309.4-second batch demonstrates automated execution time; it does not substitute for a human-effort measurement or semantic review.

E. Results and audit

All nine jobs passed every acceptance check. Their greedy yields are the “greedy” rows of Table IV, and they appear unchanged in Fig. 5(c,d). Only 1 of 600 chips was fail-free before repair, so chip yield gain is within 0.2 points of chip yield. Taken at face value, the greedy results favor spare rows: R128 C1, with 256 spares per region, beats R64 C2, with 320, on every wafer.

Because the harness records unrepaired regions as unresolved rather than infeasible, and keeps per-region records, these results could be audited. Defective regions hold 217–413 fails on average across 32768 rows, so nearly every action covers exactly one fail and all gains tie. Ties go to the smallest id. The generated ids begin with row for

rows and seg or segment for columns, so greedy spends row spares first: rows are 52–81 % of its selected actions. We therefore applied a column-first construction offline. In each CCR pool it takes the C columns with the most fails, covers the remaining fails with rows, and accepts the region if at most R distinct rows are needed. Each accepted region has an explicit witness satisfying (1). It repaired every region greedy repaired, plus 44–343 more per job. It raised pooled region yield by 7.9, 3.1 and 4.0 percentage points, computed from unrounded counts, and reversed the R64C2 vs. R128C1 ranking. The regions that remain unrepaired form whole high-fail chips; for example, on wafers 001 and 003 under R128C1, region and chip yield are both exactly 92.0 %, so the failed regions are 16 complete chips. The fix belongs in the solver: tie-breaking should account for pool scarcity. The point for the harness is that a heuristic failure was never reported as infeasibility, and that every input needed to check the numbers was preserved.

VIII. Lessons and Limitations

Narrow what is accepted. Restricting the agent to two model-entry functions behind a validating core, with an explicit list of unsupported constraints, lets the harness reject compilation failures, malformed models and invalid repair witnesses, so batches run unattended. It is not a correctness check: whether the model matches the architecture remains an engineering review.

Make the log the source of truth. Persisting every event before broadcasting it, and reconciling from the agent’s own session file, made reloads, crashes and cancellations uneventful. Marking interrupted work instead of replaying it avoided duplicate model charges.

Share expensive steps. Code generation dominated wall time (93.5 %). Sharing one program per architecture across wafers is the largest saving; concurrent coding runs would be the next.

Reinvest released engineering effort. Under the assumptions of Section VII-D, the harness cuts estimated implementation and evaluation effort by 90.3 %, releasing almost seven engineer-days in the three-configuration study. The useful development effect is broader design exploration and more time for semantic review, rather than removing the engineer from the decision process.

Scope reads, not just writes. The agent read files outside its workspace. A future version should run agent tools in an OS-level sandbox (a container or a restricted user with a read-only view of the needed files) instead of relying on instructions. Process limits today contain mistakes; they do not isolate a malicious agent.

Other limitations. The run limits apply per process. The worker is single-threaded. All evaluation data are synthetic and use 200-chip wafers. Greedy and column-first yields are lower bounds, and HiGHS has not yet been run on these inputs. Testing is manual.

IX. Conclusion

The harness lets a user go from a list of architectures and wafers to saved, comparable conclusions with one submission, while the agent writes the architecture-specific code. Its central choices are supervised process trees with explicit deadlines, a durable event log with reconciliation and non-replaying recovery, a single queue that shares code generation across wafers, and acceptance only after protocol, invariant and integrity checks. A recorded nine-job batch completed in about five minutes, mostly spent on code generation, and its preserved per-region records made it possible to improve region yield lower bounds by up to 10.7 percentage points. Under a 100-lines-per-engineer-day baseline and a two-hour assisted allowance per configuration, the three configurations release an estimated 55.76 engineer-hours (90.3 %) and support $10.3\times$ as many configuration evaluations at equal staffing. These conditional savings can accelerate architecture exploration and free engineering effort for model review; they are separate from the measured execution time. Next steps are OS-level sandboxing of agent tools, concurrent coding runs, scarcity-aware tie-breaking, and exact HiGHS runs inside the same pipeline.

Acknowledgment

The interface font is Fusion Pixel 12px [11], used under its open license.

References

- [1] S.-Y. Kuo and W. K. Fuchs, “Efficient spare allocation for reconfigurable arrays,” *IEEE Design & Test of Computers*, vol. 4, no. 1, pp. 24–31, 1987.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- [4] WHATWG, “HTML living standard, section 9.2: Server-sent events,” <https://html.spec.whatwg.org/multipage/server-sent-events.html>.
- [5] C. H. Stapper, “On yield, fault distributions, and clustering of particles,” *IBM Journal of Research and Development*, vol. 30, no. 3, pp. 326–338, 1986.
- [6] S. J. Bae, J. Y. Hwang, and W. Kuo, “Yield prediction via spatial modeling of clustered defect counts across a wafer map,” *IEEE Transactions*, vol. 39, no. 12, pp. 1073–1083, 2007.
- [7] M.-J. Wu, J.-S. R. Jang, and J.-L. Chen, “Wafer map failure pattern recognition and similarity ranking for large-scale data sets,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 28, no. 1, pp. 1–12, 2015.
- [8] PixiJS contributors, “PixiJS v8: 2d rendering engine for the web,” <https://pixijs.com>.
- [9] D. S. Johnson, “Approximation algorithms for combinatorial problems,” *Journal of Computer and System Sciences*, vol. 9, no. 3, pp. 256–278, 1974.
- [10] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.
- [11] TakWolf, “Fusion pixel font, 12px proportional (sil open font license 1.1),” <https://github.com/TakWolf/fusion-pixel-font>.